Day 172 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 5 reference

Watch: Cross-Validation for Time Series Forecasting (Egor Howell) | Your Random Train-Test Split May Be Training on the Future (Simply Machine Learning)

`day172_cv_time_series_grouped.py`, segment by segment, in the order `main()` uses things. Today's new parts are shuffled_folds, blocked_folds, forward_folds, group_folds and cross_val, copied unchanged into day172_exercise.py; Day 109's boosted trees, Day 147's series loaders and Week 23's MovieLens loaders are imported. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: n = rows · k = folds · tr, te = training and test row numbers · t, T = each row's month · o = a forward origin · H = horizon (12) · M = the MovieLens arrays · dev, new = rows of fold users and of never-seen users

Segment 0 — Setup

Imports and constants

CODE

```
for w in ("week-16-classical-ml-trees", "week-22-time-series-analysis",
    "week-23-recommender-systems"):
    sys.path.insert(0, os.path.join(ROOT, w))
with contextlib.redirect_stdout(io.StringIO()):
    import day109_exercise as d109
    import day147_exercise as d147
    import day154_exercise as d154
    import day156_exercise as d156
    import day158_exercise as d158
H = 12
LAGS = 12
K = 5
HOLD_MONTHS = 60
SIMS, SIM_N, FUTURES = 50, 300, 400
DEPTHS = (1, 2, 4, 6, 8)
ROUNDS, RATE, MIN_LEAF = 50, 0.2, 20
NEW_USERS, DEV_USERS = 300, 400
CV_SEEDS = 2
```

- `for w in (...): sys.path.insert(0, ...)` — three earlier weeks' folders on the import path in one loop.
- `SIMS, SIM_N, FUTURES = 50, 300, 400` — three constants by tuple unpacking.
- `DEPTHS = (1, 2, 4, 6, 8)` — the tree depths each scheme will choose between.

Segment 1 — The parts

shuffled_folds and blocked_folds

MATH

$$block(i) = floor(i\ k / n) \quad train = rows\ outside\ [first\ test - gap,\ last\ test + gap]$$

CODE

```
def shuffled_folds(n, k, seed):
    f = np.random.RandomState(seed).permutation(n) % k
    return [(np.flatnonzero(f != j), np.flatnonzero(f == j)) for j in range(k)]
def blocked_folds(n, k, gap=0):
    b = np.arange(n) * k // n
    out = []
    for j in range(k):
        te = np.flatnonzero(b == j); lo, hi = te[0] - gap, te[-1] + gap
        tr = np.flatnonzero((b != j) & ((np.arange(n) < lo) | (np.arange(n) > hi)))
        out.append((tr, te))
    return out
```

- `permutation(n) % k` — a random order, then 0, 1, ..., k-1, 0, 1, ...: equal-sized random folds.
- `np.flatnonzero(f != j)` — the row numbers where the condition holds.
- `np.arange(n) * k // n` — integer division gives each row its block, in order: 0 for the first n/k rows, and so on.
- `te[0] - gap, te[-1] + gap` — the first and last test rows, widened by the gap.
- `(b != j) & ((...) | (...))` — & and | combine boolean arrays element by element; brackets are required.

forward_folds, group_folds and cross_val

MATH

$$CV = (1/k)\ \Sigma_j\ score(fit\ on\ train_j,\ test_j)$$

CODE

```
def forward_folds(t, origins, horizon):
    t = np.asarray(t)
    return [(np.flatnonzero(t <= o), np.flatnonzero((t > o) & (t <= o + horizon))) for o in
        origins]
def group_folds(groups, k, seed):
    g = np.unique(groups); f = dict(zip(g, np.random.RandomState(seed).permutation(len(g)) % k))
    fold = np.array([f[x] for x in groups])
    return [(np.flatnonzero(fold != j), np.flatnonzero(fold == j)) for j in range(k)]
def cross_val(folds, fit_score):
    s = np.array([fit_score(tr, te) for tr, te in folds])
    return s.mean(), s
```

- `(t > o) & (t <= o + horizon)` — the next `horizon` steps after the origin.
- `dict(zip(g, ...))` — pairs each group with a fold number, as a dictionary.
- `[f[x] for x in groups]` — each row looks up its group's fold.
- `fit_score(tr, te) for tr, te in folds` — each pair is unpacked in the loop header; the function does the rest.
- `return s.mean(), s` — the estimate, and every fold's score so they can be inspected.

design and fit_predict: three forecasters

CODE

```
def design(y, kind):
    n = len(y); t = np.arange(n)
    if kind == "lag":
        T = np.arange(H + LAGS - 1, n)
        X = np.column_stack([y[T - H - j] for j in range(LAGS)] + [np.ones(len(T))])
        return X, y[T], T
    X = np.column_stack([t / 100, (t / 100) ** 2] + [(t % 12 == m).astype(float) for m in
        range(12)])
    return X, y.copy(), t
def fit_predict(kind, X, y, t, tr, te):
    if kind == "knn":
        nn = np.argsort(np.abs(t[te][:, None] - t[tr][None, :]), 1)[:, :3]
        return y[tr][nn].mean(1)
    w = np.linalg.lstsq(X[tr], y[tr], rcond=None)[0]
    return X[te] @ w
```

- `np.column_stack([y[T - H - j] for j in range(LAGS)] + [np.ones(len(T))])` — 12 lagged columns plus a column of ones (the intercept), side by side.
- `(t % 12 == m).astype(float)` — 1 in calendar month m, 0 elsewhere: a dummy column per month.
- `np.argsort(np.abs(t[te][:, None] - t[tr][None, :]), 1)[:, :3]` — distances from every test month to every training month; the 3 closest.
- `y[tr][nn].mean(1)` — their mean, one value per test month.
- `np.linalg.lstsq(X[tr], y[tr], rcond=None)[0]` — least-squares weights; [0] drops the residuals and rank it also returns.

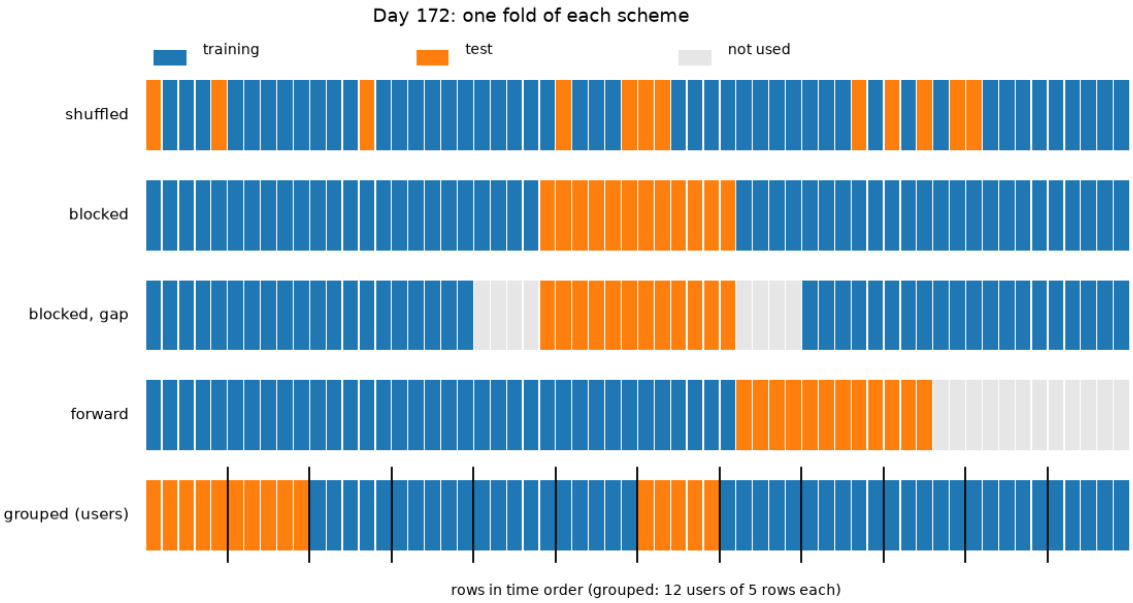
schemes: four ways to cut the same rows

CODE

```
def schemes(n_rows, t, last):
    origins = range(last - 5 * H, last - H + 1, H)
    return {"shuffled": shuffled_folds(n_rows, K, 0), "blocked": blocked_folds(n_rows, K),
            f"blocked, gap {H + LAGS}": blocked_folds(n_rows, K, gap=H + LAGS),
            "forward": forward_folds(t, origins, H)}
```

- `range(last - 5 * H, last - H + 1, H)` — five origins a year apart; the last leaves exactly 12 months to test.
- `f"blocked, gap {H + LAGS}"` — an f-string used as a dictionary key.
- `{...: ..., ...}` — a dictionary of name → list of folds, so every scheme is run the same way.

SYSTEM DIAGRAM



One fold of each scheme on 60 rows.

Segment 2 — The experiments

simulate and true_error: a known truth

MATH

$$level_t = level_{t-1} + 0.05 + 0.3\,\varepsilon_t \quad y_t = level_t + 2\sin(2\pi t/12) + 0.5\,\eta_t$$

CODE

```
def simulate(rng, n):
    level = np.cumsum(0.05 + 0.3 * rng.randn(n))
    return level + 2 * np.sin(2 * np.pi * np.arange(n) / 12) + 0.5 * rng.randn(n), level[-1]
def true_error(kind, y, last_level, rng):
    n = len(y); X, Y, T = design(y, kind)
    tf = np.arange(n, n + H)
    steps = 0.05 + 0.3 * rng.randn(FUTURES, H)
    fut = last_level + np.cumsum(steps, 1) + 2 * np.sin(2 * np.pi * tf / 12) + 0.5 *
        rng.randn(FUTURES, H)
    if kind == "lag":
        Xf = np.column_stack([y[tf - H - j] for j in range(LAGS)] + [np.ones(H)])
        w = np.linalg.lstsq(X, Y, rcond=None)[0]; p = Xf @ w
    elif kind == "knn":
        p = np.full(H, y[-3:].mean())
    else:
        Xf = np.column_stack([tf / 100, (tf / 100) ** 2] + [(tf % 12 == m).astype(float) for m in
            range(12)])
        p = Xf @ np.linalg.lstsq(X, Y, rcond=None)[0]
    return np.mean(np.abs(fut - p))
```

- `np.cumsum(0.05 + 0.3 * rng.randn(n))` — a running sum of random steps with a small upward drift: a random walk.
- `np.cumsum(steps, 1)` — the running sum along each row: 400 futures at once.

- `last_level + ...` — every future starts from the series' true last level, known only because it is simulated.
- `np.full(H, y[-3:].mean())` — knn's forecast: the mean of the last 3 months, for all 12 future months.

demo_simulated: estimates against the truth

CODE

```
for s in range(SIMS):
    y, lv = simulate(rng, SIM_N)
    for kind in KINDS:
        X, Y, T = design(y, kind)
        for name, folds in schemes(len(Y), T, SIM_N - 1).items():
            out[kind].setdefault(name, []).append(cross_val(folds, mae_scorer(kind, X, Y, T))[0])
        out[kind].setdefault("truth", []).append(true_error(kind, y, lv, rng))
```

- `out[kind].setdefault(name, []).append(...)` — setdefault returns the list for that key, creating it on first use.
- `cross_val(...)[0]` — only the mean is kept here.
- `min(KINDS, key=lambda k: ...)` — the forecaster with the smallest estimate: what each scheme would choose.

movielens and ml_features: no second leak

CODE

```
def ml_features(M, rows, fit_rows, use_time=True, use_user=True):
    ni = M["i"].max() + 1; g = M["r"][fit_rows].mean()
    s = np.bincount(M["i"][fit_rows], M["r"][fit_rows], ni); c = np.bincount(M["i"][fit_rows],
        minlength=ni)
    film = (s + 10 * g) / (c + 10)
    i, u = M["i"][rows], M["u"][rows]
    cols = [film[i], c[i], M["yr"][i]] + ([M["age"][u], M["fem"][u], M["occ"][u]] if use_user else
        [])
    return np.column_stack(cols + ([M["ts"][rows]] if use_time else []))
```

- `np.bincount(M["i"][fit_rows], M["r"][fit_rows], ni)` — sums of ratings per film, from the training rows only.
- `(s + 10 * g) / (c + 10)` — each film's mean pulled towards the overall mean, as if it had 10 extra average ratings.
- `[...] + ([...] if use_user else [])` — a conditional list: the user columns only when asked for.
- `film[i]` — fancy indexing: each row gets its own film's value.

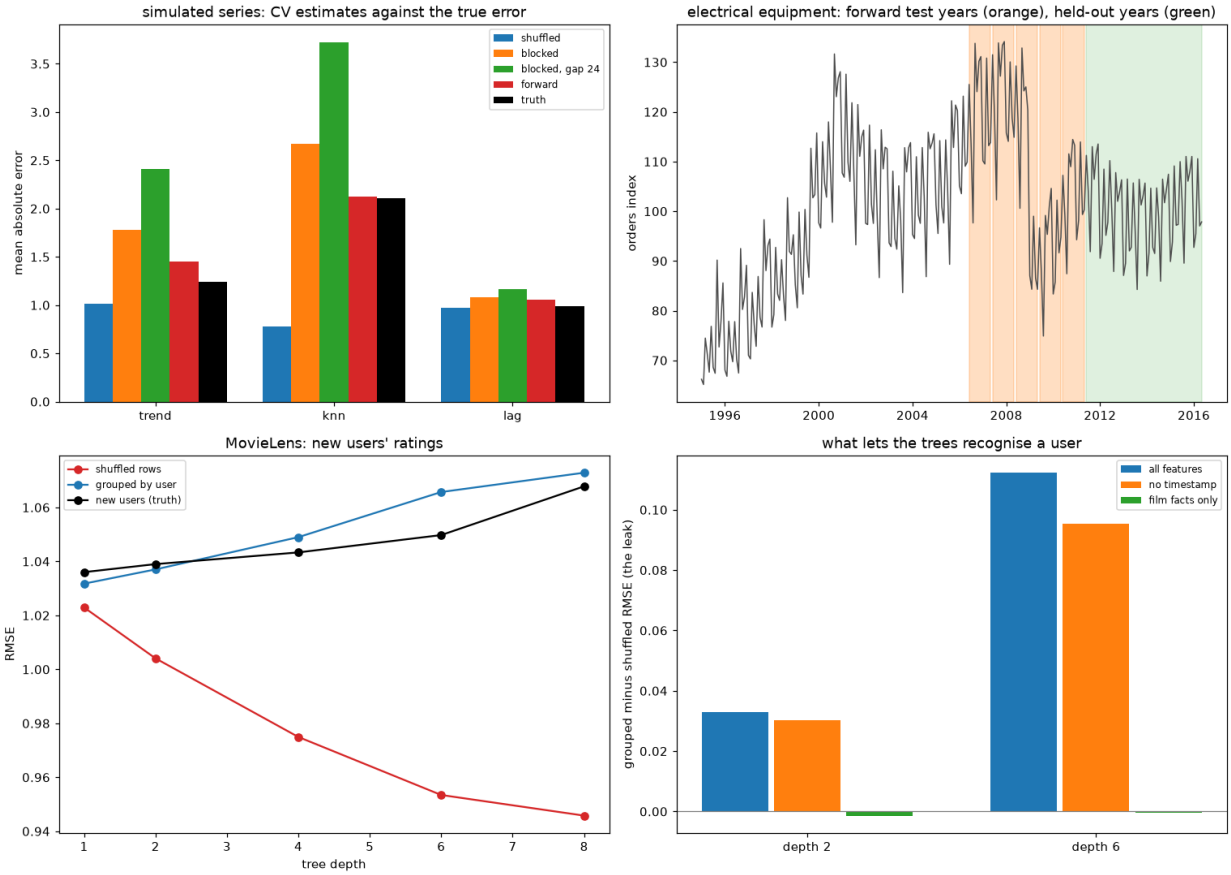
ml_estimates: shuffled, grouped and the truth

CODE

```
def ml_estimates(M, depth, use_time=True, use_user=True):
    dev = M["dev"]; fs = ml_scorer(M, depth, use_time, use_user)
    absf = lambda folds: [(dev[a], dev[b]) for a, b in folds]
    sh = np.mean([cross_val(absf(shuffled_folds(len(dev), K, s)), fs)[0] for s in
        range(CV_SEEDS)])
    gr = np.mean([cross_val(absf(group_folds(M["u"][dev], K, s)), fs)[0] for s in
        range(CV_SEEDS)])
    return sh, gr, fs(dev, M["new"])
```

- `absf = lambda folds: [(dev[a], dev[b]) for a, b in folds]` — the splitters number rows 0..len(dev)-1; this turns them into MovieLens row numbers.
- `group_folds(M["u"][dev], K, s)` — the user of each dev row is its group.
- `fs(dev, M["new"])` — the same scorer, fitted on every dev row and scored on the new users.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.arange(n) * k // n` — contiguous equal blocks.
- `a & b, a | b` — element-wise and / or of boolean arrays.
- `dict(zip(keys, values))` — a dictionary from two sequences.
- `d.setdefault(k, []).append(x)` — append to a list that may not exist yet.
- `np.bincount(idx, weights, n)` — sums of weights per index.
- `np.linalg.lstsq(X, y, rcond=None)[0]` — least-squares weights.
- `min(items, key=f)` — the item with the smallest f(item).

End of Day 172 syntax reference. For the experiments and what they showed, see the Day 172 complete guide.